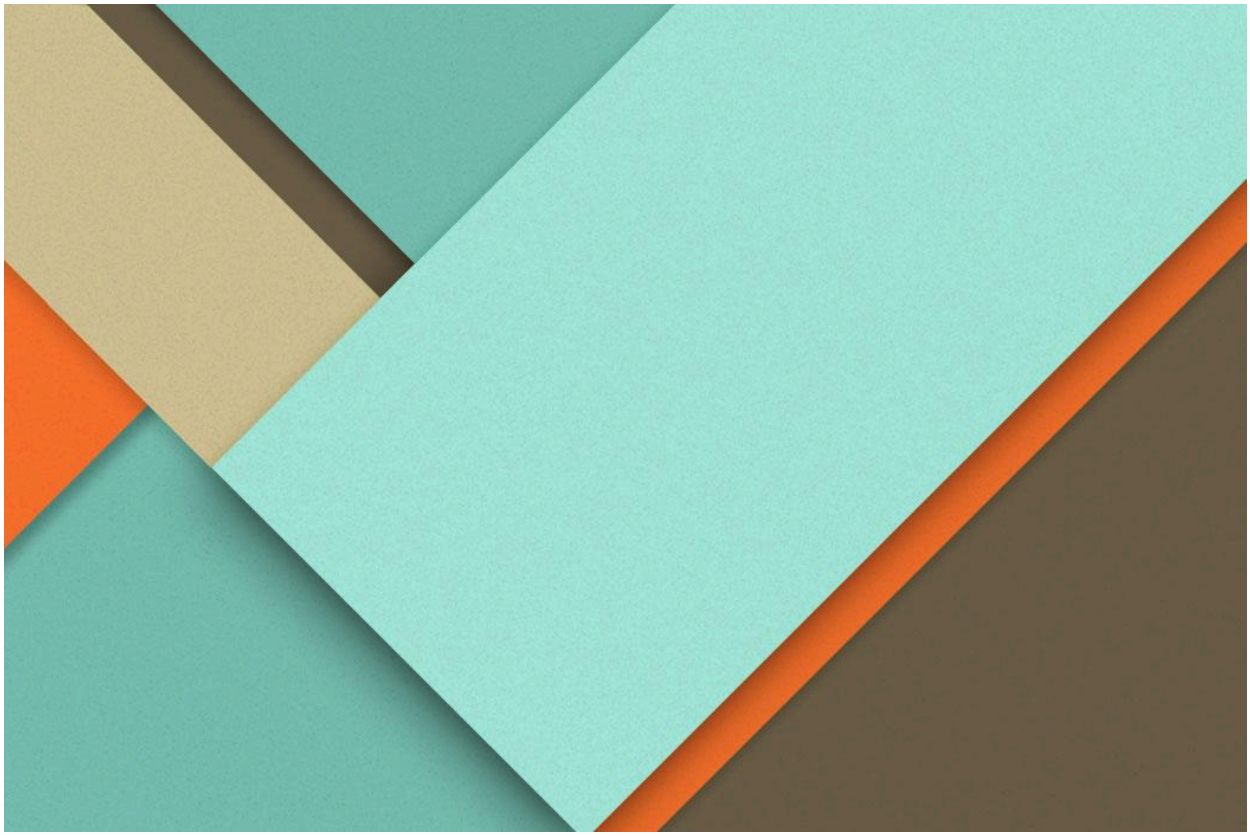


Interactive Prototype Specification

10.29.2026

—

Logan Smith

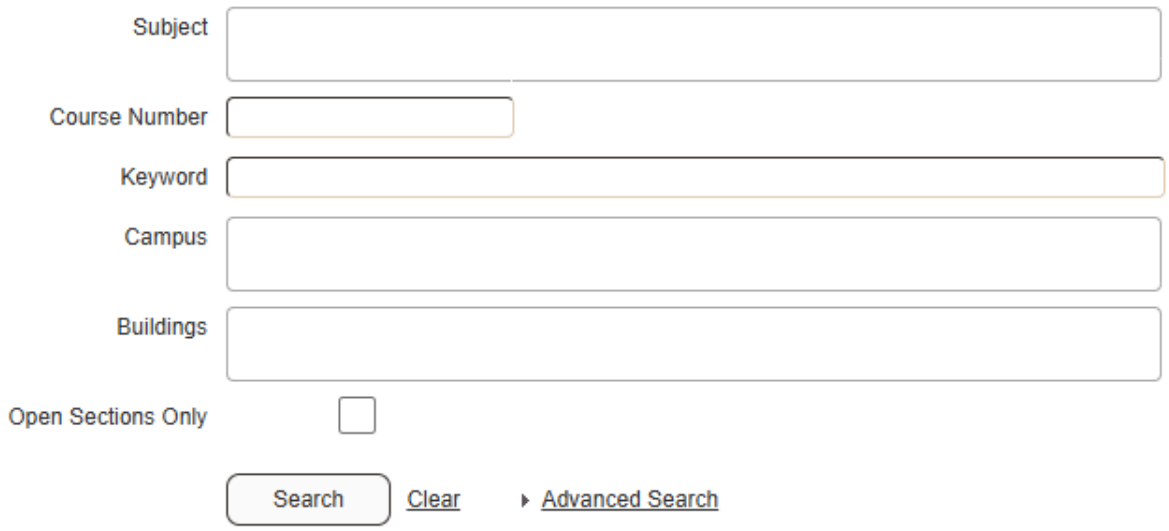
Github Repo URL: https://github.com/ljswebdev/interactive-prototype-specification

Live Prototype: https://ljswebdev.github.io/interactive-prototype-specification/

Interaction Mechanics and Input Paradigm Mapping

User Flow Selection

My target system will be Weber State University's registration system's search input. I feel that it is the central part of the class registration system, as it allows students to search for and browse classes as they build their prospective semester class schedules.



Input Mechanics Analysis

My target system will be Weber State University's registration system's search input. I feel that it is the central part of the class registration system, as it allows students to search for and browse classes as they build their prospective semester class schedules.

The primary input paradigm for the search input is point-and-click, and the initial screen displays the following distinctive UI elements:

- 3 autocomplete dropdowns (Subject, Campus and Buildings)
- 2 text inputs (Course Number and Keyword)
- 1 checkbox (Open Sections Only
- 1 button (Search)
- 2 link buttons (Clear and Advanced Search

The Advanced Search link button opens a hidden menu containing many more search parameter inputs such as Title, Instructor, ect. I will be focusing on these 9 input elements:

- Subject autocomplete dropdown: Allows users to select a subject of classes. Requires users to choose from a predefined list containing subjects such as Accounting, Computer Science, ect.
- Course Number text input: Allows users to enter a specific course number, such as 3230. Ignores special characters.
- Keyword text input: Allows users to enter keywords related to the course they are searching for, enabling flexible searching.
- Campus autocomplete dropdown: Allows users to select a specific campus location. Requires users to choose from a predefined list containing campuses such as Morgan Center, WSU Davis, WSU Ogden, ect.
- Buildings autocomplete dropdown: Allows users to further define their search by selecting a specific building on a campus. Its dropdown options do NOT filter based on the Campus autocomplete dropdown input. For example, if you select WSU Davis as the Campus, you will still see buildings that belong in the WSU Ogden campus.
- Open Sections Only checkbox: Allows users to exclude classes that are full (have no available seats)
- Search button: Submits the user's search parameters and redirects to the search results interface.
- Clear link button: Clears all of the search inputs at once, allowing users to start afresh.
- Advanced Search link button: Displays additional search inputs to allow users to further refine their search.

As the search input is a clustered collection of several different input fields, accessibility and focus indication are key HCI components that must be considered. I expect the search input to allow for and properly accommodate the three following input sources:

- Keyboard. Users must be able to navigate through all search parameters as well as submit their search inquiry with just a keyboard. This will support users who do not have or prefer not to use a mouse, trackpad or similar pointing device. As such, the Tab key as well as the Shift + Tab key combination must shift the focus through the search input parameter fields in an orderly, one-by-one manner, allowing the user to fully navigate all fields in both directions (up and down the column of inputs). Enabled text fields should accept keyboard input, enabled checkboxes should allow the user to toggle between on / off, and autocomplete inputs need to be fully operable using the arrow, Space and Enter keys.
- Screen Reader. The search input must be properly coded to accommodate screen reader technology, such as the NVDA screen reader. Each input must be properly identified by the screen reader, as well as their associated hint text and options. User input must be read back to the user by the screen reader, actions must be confirmed.
- Touch. While Weber State University's registration system works best on a PC or laptop device, the search input must operate in a similar fashion on a mobile device. The search input elements must be properly resized using correct CSS media queries and allow users to select and enter input without accident. The autocomplete input text fields must open to the size of the device's width to prevent screen overflow and clipped UI.

Fitts' Law Target Sizing

I will be applying the rule that the minimum size of a button cannot be less than 44px by 44px - that is, elements cannot be less than 44px in height nor less than 44px wide.

The three autocomplete inputs (Subject, Campus and Buildings) are 500px by 40px. These elements are plenty wide, but fall under the minimum height rule by 4px.

The two text input fields (Course Number and Keyword) are 154px by 22px and 502px by 22px respectively. They are both significantly below the minimum height rule.

The Open Sections Only checkbox similarly has a clickable area of 140px by 20px, which also breaks the minimum height rule.

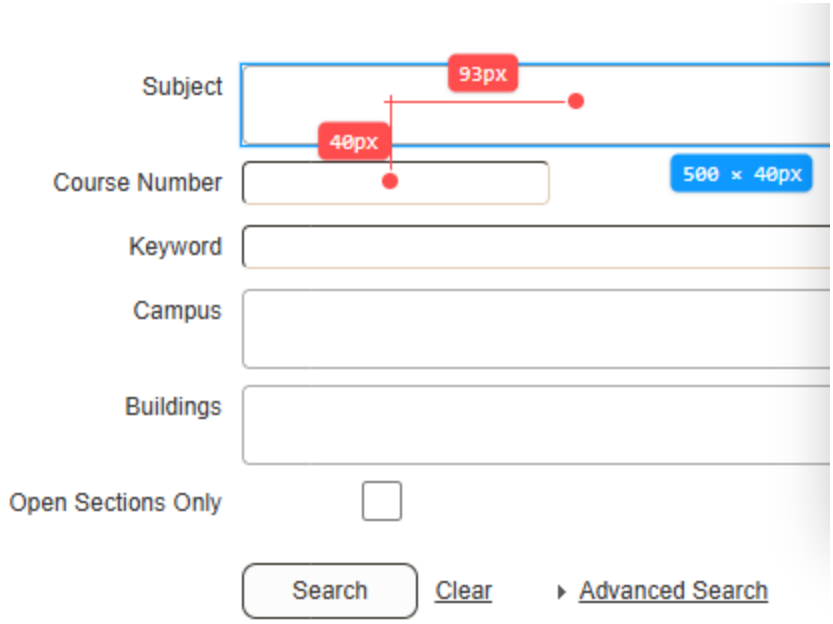
The Search button has a size of 88px by 20px, breaking the minimum height rule.

The Clear link button breaks both the established width and height rules, as its size is a mere 28px by 14px. It also breaks the minimum height rule by 30px.

And finally, the Advanced Search button link is sized at 109px by 14px, breaking the minimum height rule by 30px.

I will be systematically calculating Fitts' Law as the user navigates through the first Subject autocomplete dropdown to the final Search button, simulating a user adding input to each parameter and performing a search.

Subject → Course Number



The distance D between clicking on Subject and then clicking on Course Number is approximately 101.24px by calculating the square root of 93^2 + 40^2.

The width W of Course Number is 154px

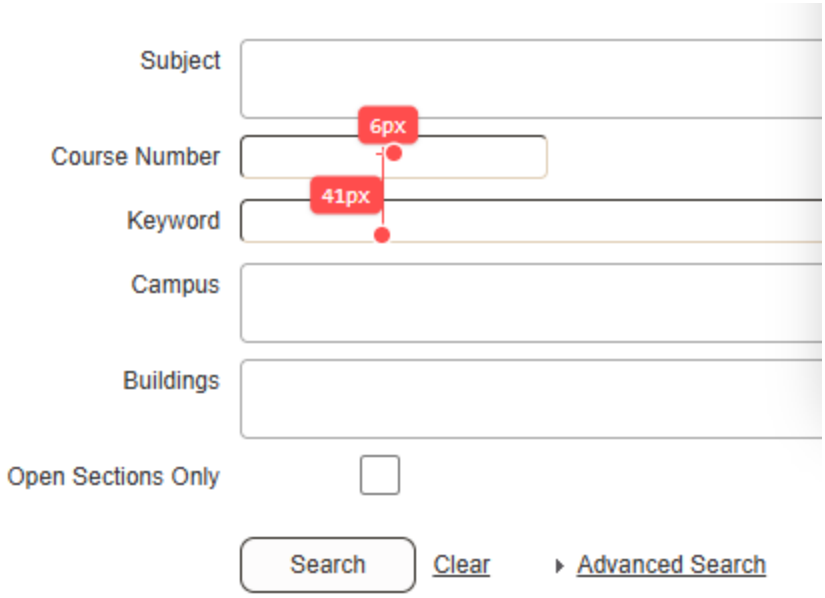
The Index of Difficulty ID is found by the Fitts' Law equation below:

ID = log2((101.2 / 154) + 1)

ID = 0.72

An index of difficulty of 0.72 is a relatively low difficulty due to the width of Course Number, and the assumption that the user will approach the Course Number element from a left horizontal angle.

Course Number → Keyword



The distance D between clicking on Keyword after clicking on Course Number is approximately 41.44px by calculating the square root of 6^2 + 41^2.

Granted that this is a strongly vertical motion, we will use the height of Keyword in our equation rather than width. So, The height W of Keyword is 22px

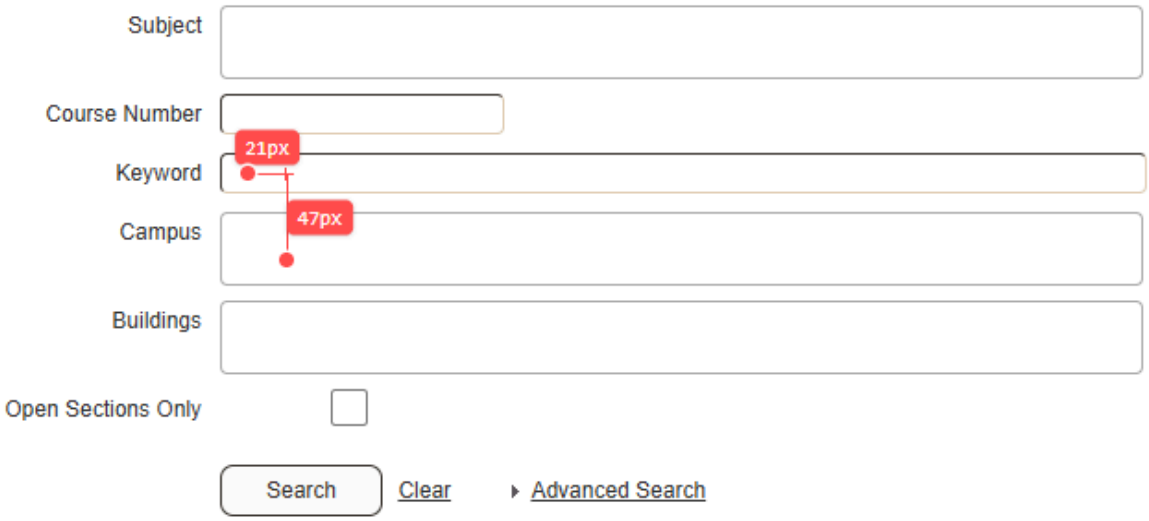
The Index of Difficulty ID is found by the Fitts' Law equation below:

ID = log2((41.44 / 22) + 1)

ID = 1.53

An index of difficulty of 0.72 is a relatively low difficulty due to the width of Course Number, and the assumption that the user will approach the Course Number element from a left horizontal angle.

Keyword → Campus



The distance D between clicking on Campus after clicking on Keyword is approximately 51.48px by calculating the square root of 21^2 + 47^2.

This is a mostly vertical motion moving downward at an angle, so height W will be our variable. The height W of Campus is 40px

The Index of Difficulty ID is found by the Fitts' Law equation below:

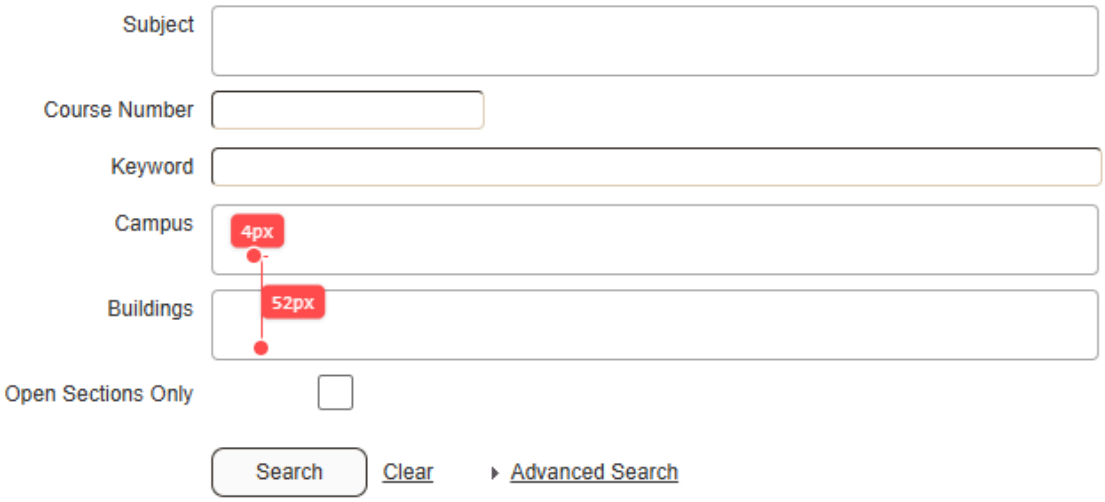
ID = log2((51.48 / 40) + 1)

ID = 1.25

This is a slightly easier target as Campus is nearly double the height of Keyword, and the reduced index of difficulty reflects that.

Campus → Buildings

The distance D between clicking on Buildings after clicking on Campus is approximately 52.15px by calculating the square root of 4^2 + 52^2.



The movement is almost perfectly vertical, so we will go with the building input field's height H = 40, which is the same as Campus's height.

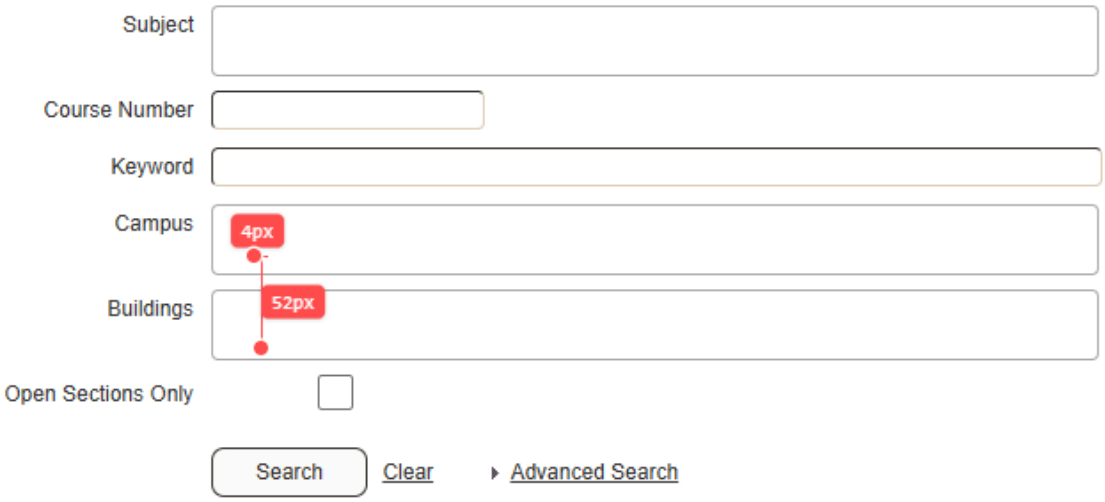
The Index of Difficulty ID is found by the Fitts' Law equation below:

ID = log2((52.15 / 40) + 1)

ID = 1.20

The move from Campus to Buildings is even slightly easier than the previous Keyword to Campus jump due to Building's height of 40px.

Buildings → Open Sections Only Checkbox



The distance D between clicking on the Open Sections Only checkbox after clicking on Buildings is approximately 66.21px by calculating the square root of 60^2 + 28^2.

Given that this movement is largely horizontal, we will go with the the width of the Open Sections Only checkbox (the clickable area, not the visible checkbox itself) which will set H = 140.

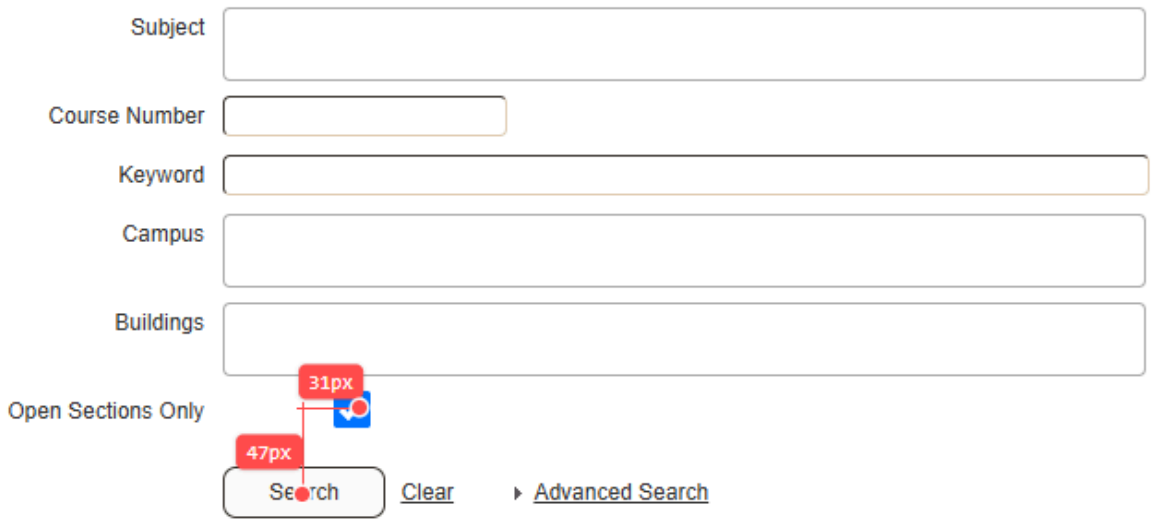
The Index of Difficulty ID is found by the Fitts' Law equation below:

ID = log2((66.21 / 140) + 1)

ID = 0.56

This the lowest index of difficulty thus far, granted that it is a largely horizontal motion and we are accounting for Open Section Only's clickable area width of 140px.

Open Sections Only Checkbox → Search



The distance D between clicking on the Search button after clicking on the Open Sections Only checkbox is approximately 56.30px by calculating the square root of 31^2 + 47^2.

We will go with the Search bar's height of H = 28 given that the movement is predominately vertical.

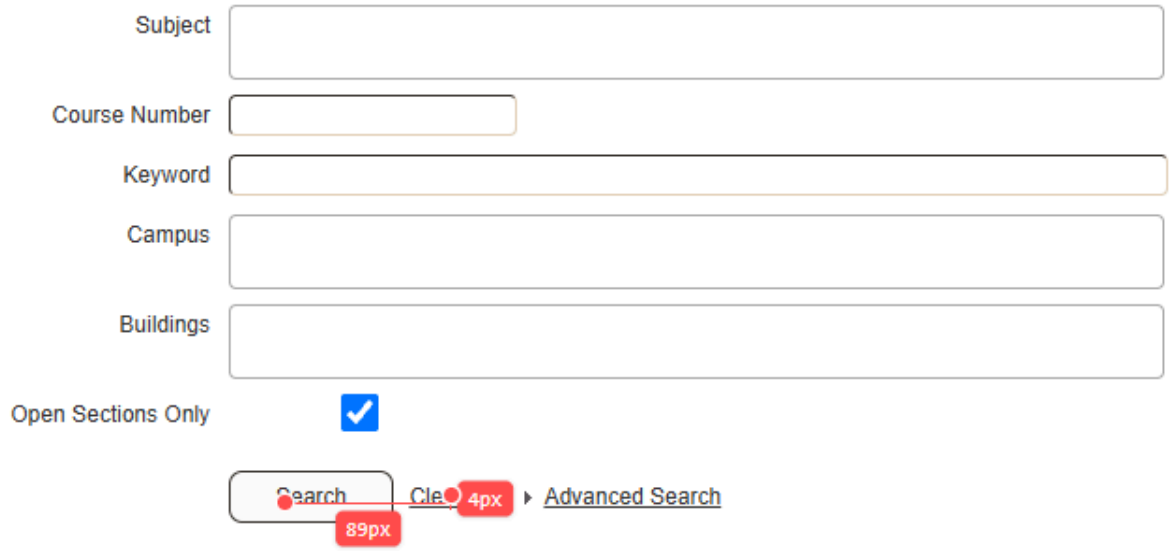
The Index of Difficulty ID is found by the Fitts' Law equation below:

ID = log2((56.30 / 28) + 1)

ID = 1.59

This is a slightly more difficult maneuver than Campus to Building's index of difficulty (1.20) but not particularly.

Search → Clear



The distance D between clicking on the Clear link button after clicking on the Search button is approximately 89.09px by calculating the square root of 89^2 + 4^2.

Seeing that this is an overwhelmingly vertical movement, we will rely on the width of the Clear link, which is to say that H = 28

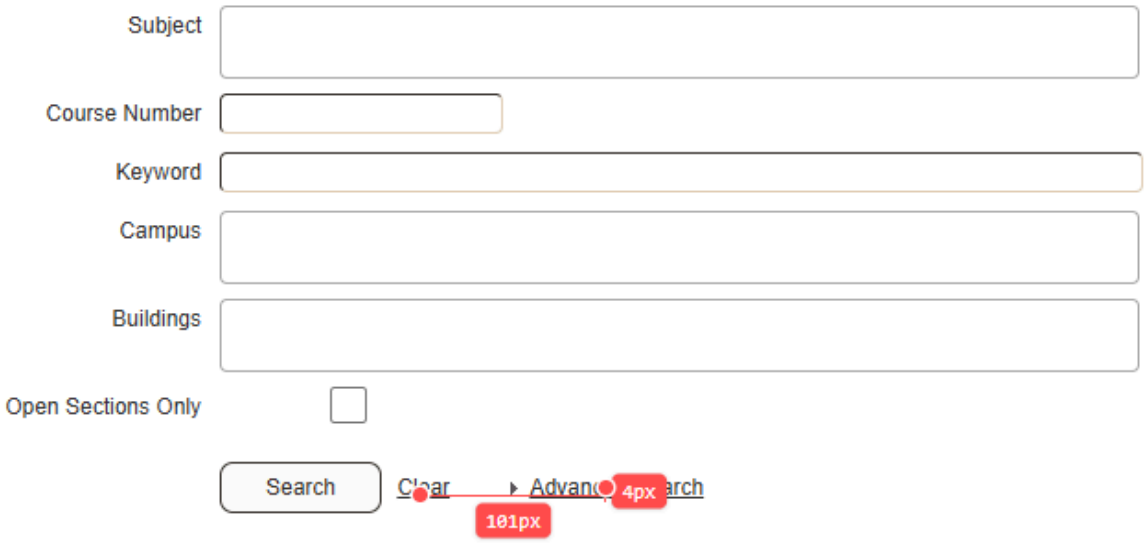
The Index of Difficulty ID is found by the Fitts' Law equation below:

ID = log2((89.09 / 28) + 1)

ID = 2.04

This a much tougher maneuver as even though it is nearly entirely horizontal, the width of the Clear link button is only 28px, giving us a much smaller target to land on.

Clear → Advanced Search



The distance D between clicking on the Advanced Search link button after clicking on the Clear link button is approximately 101.08px by calculating the square root of 101^2 + 4^2.

As this last movement is another mostly horizontal maneuver, we will go with the width of the Advanced Search link button as H = 109px

The Index of Difficulty ID is found by the Fitts' Law equation below:

ID = log2((101.08 / 109) + 1)

ID = 0.95

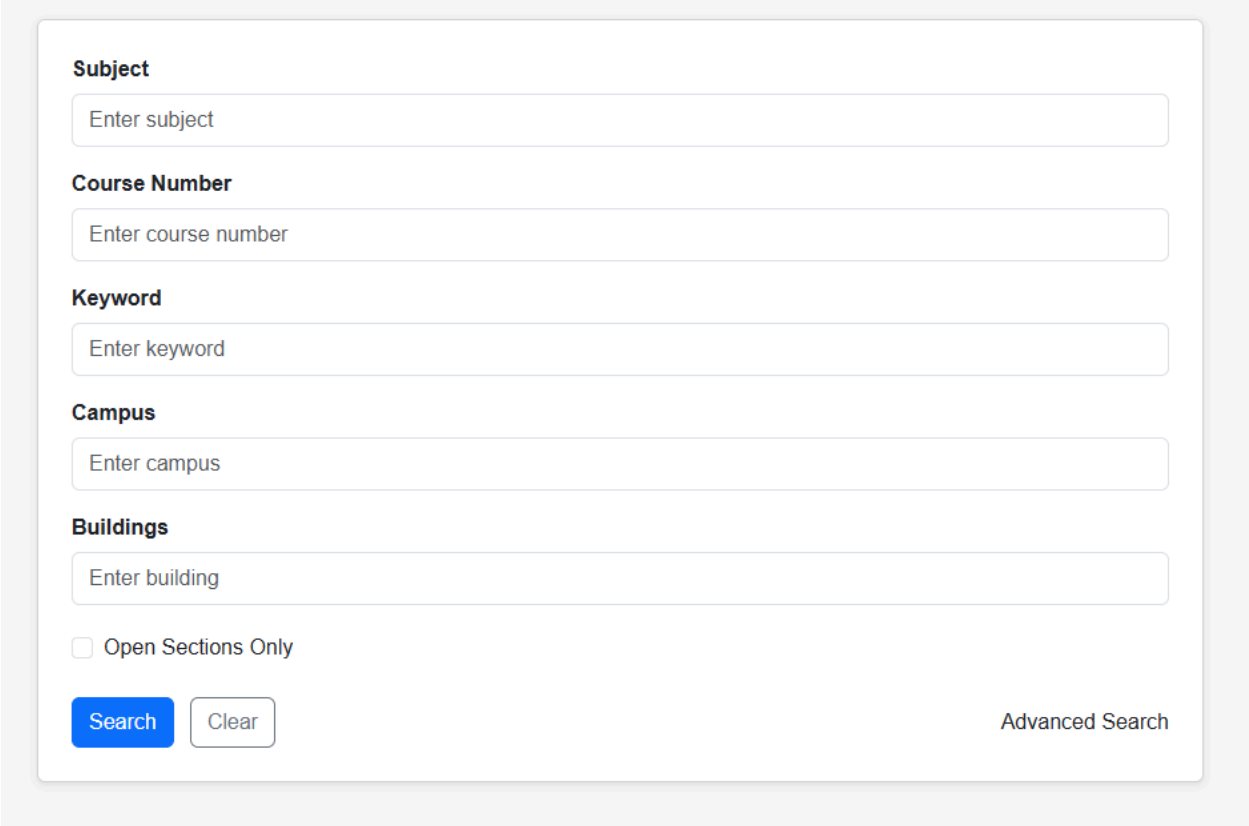
This is another relatively simpler maneuver granted that the width of the Advanced Search input button is a fairly wide target.

Summary

These Fitts' law calculations show us that the most difficult click-and-point movement is from the Subject autocomplete dropdown to the Course Number input with an index of difficulty at 2.49, and the easiest maneuver is the Buildings autocomplete dropdown to the Open Sections Only checkbox with an index of difficulty of 0.56.

Fitts' Law Target Sizing

I have created a prototype using HTML5, CSS3, Javascript and Bootstrap 5 components meant to be an HCI optimized version of Weber State's registration system search input.



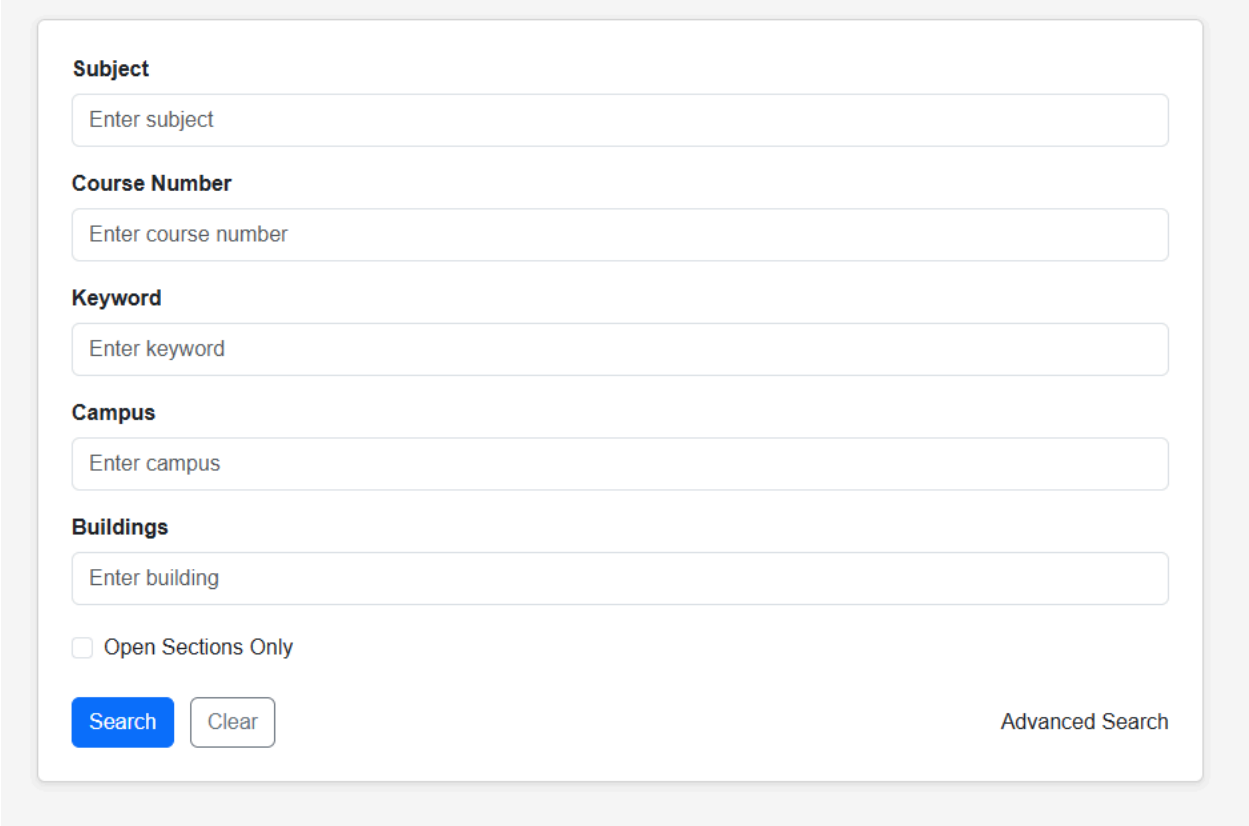
I will now be examining and documenting how this prototype behaves under each of the following visual and programmatic states:

- Default / Resting
- Hover / Pointer-Over
- Focus
- Active / Pressed
- Disabled
- Loading / System Processing
- Error / Validation Alert

I will example each individual component type, as they can be categorized into the five following categories: autocomplete inputs, text inputs, buttons, checkboxes and link buttons (buttons that are simply text, such as Advanced Search)

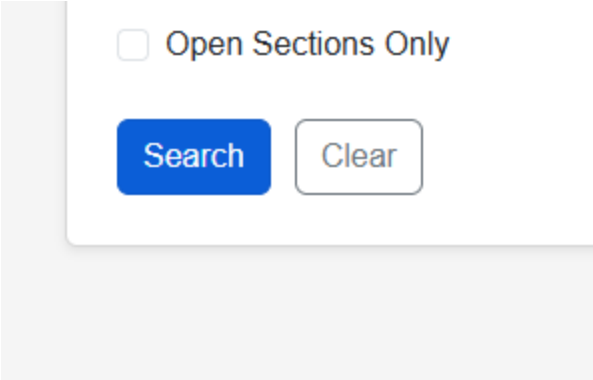
Default / Resting state

These are each of the categorized inputs in their default / resting state and the prototype's initial appearance upon load. Each of the inputs is clearly labeled with bold text uniformly placed above each input, each input has the same proper amount of spacing between them, and the Search button is colored in Bootstrap's primary blue to provide a start contrast and call-to-action polish. The Clear button is colored in a secondary grey outline, indicating that it is a functional but optional button, and the Advanced Search link button is conveniently placed in the bottom-right corner to serve as an open / close button for the additional search parameter fields.

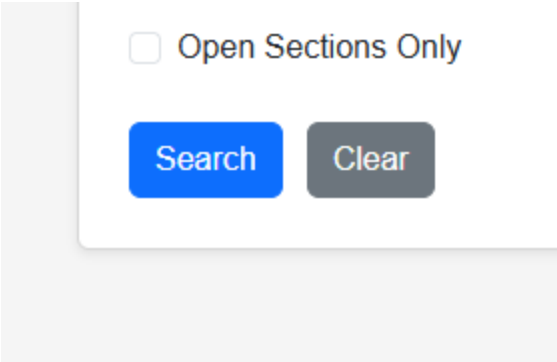


Hover / Pointer State

The Search button changes to a subtle shade of blue upon hover, indicating that the pointer has entered its clickable area.

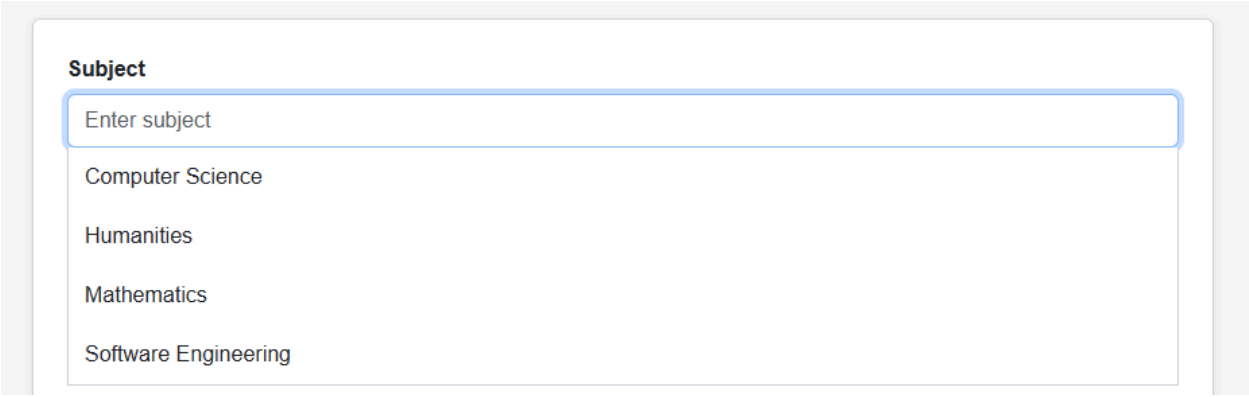


The Search button changes to a subtle shade of blue upon hover, indicating that the pointer has entered its clickable area.

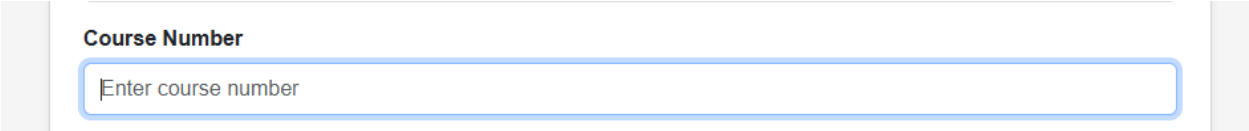


Focus

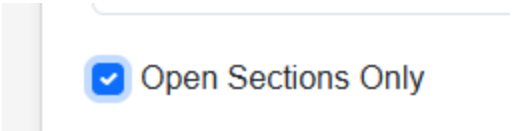
The autocomplete inputs (such as Subject) gain a light blue thick outline around the input, and the drop-down menu opens and displays the respective options (Computer Science, Humanities, Mathematics, ect...)



The text inputs (such as Course Number) gain a light blue thick outline that matches each of the other inputs' focus states.

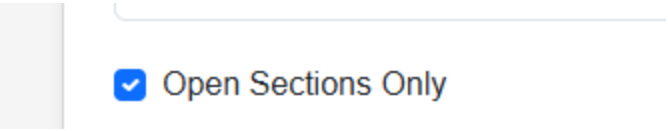


The checkbox inputs (such as Open Sections Only) gain a similar light blue thick outline around the border of the checkbox.

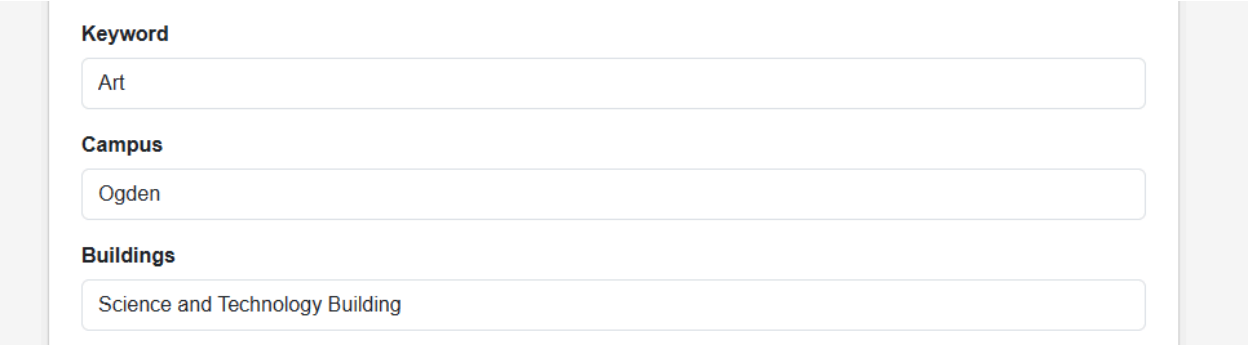


Active / Pressed

The checkbox inputs (such as Open Sections Only) show a clear white check mark within a Bootstrap primary blue box to indicate that the checkbox is checked, and the Open Sections Only filter has been applied.



The autocomplete and text inputs (such as Campus / Buildings and Keyword respectively) clearly show the selected or typed in input within the input field.



Disabled

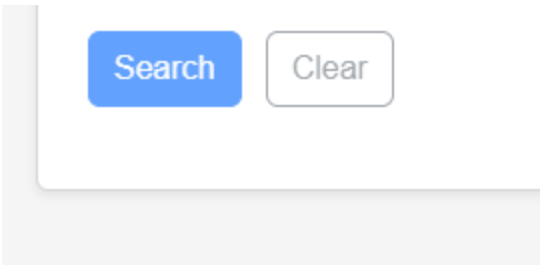
The autocomplete and text inputs are greyed out in their disabled states and do not allow clicking.



The checkboxes (Open Sections Only) similarly have both their text and the checkbox greyed out.



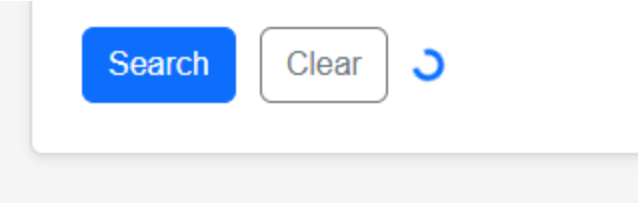
The Search and Clear buttons have a slight but noticeable layer of transparency overlay.



In the registration system search input's current form, there is no need to disable any of the fields, so these disabled displays are for demonstration / example purposes only.

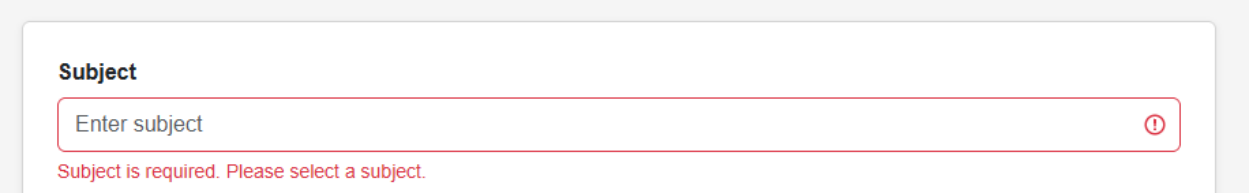
Loading / System Processing

After clicking the Search button, a blue animated loading spinner appears next to the Clear button. The spinner appears immediately and spins for the duration of the loading process, and then vanishes after the search function completes.

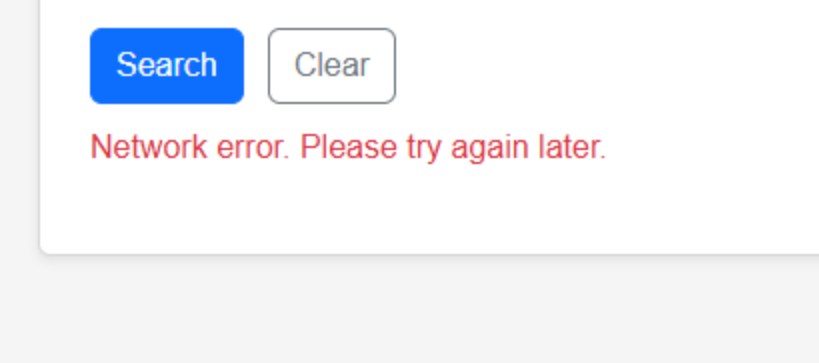


Error / Validation Alert

The Subject autocomplete input gains a thin but highly contrasting red outline upon error (For example purposes, the subject filter is required) and it also displays red text beneath the input that reads “Subject is required. Please select a subject.” There is also a small red warning icon on the right side of the input field.



If there is a network connection error, then red text appears beneath the Search button that reads “Network error. Please try again later.”



Accessibility Architecture and WCAG Specification

Semantic HTML markup

The high-level HTML markup structure for my proposed each input prototype is as follows:

```
<main>
  <section>
    <form>
      <div>
        <label>Subject</label>
        <input>
      </div>

      <div>
        <label>Course Number</label>
        <input>
      </div>

      <div>
        <label>Keyword</label>
        <input>
      </div>

      <div>
        <label>Campus</label>
        <input>
      </div>
```

```
        <div>
          <label>Buildings</label>
          <input>
        </div>

        <fieldset>
          <legend>Section Availability</legend>
          <input type="checkbox">
          <label>Open Sections Only</label>
        </fieldset>

        <button>Search</button>
        <button>Clear</button>
      </form>
    </section>
</main>
```

Keyboard Navigation and Interaction

My proposed prototype supports full keyboard navigation for users that are unable or do not wish to use a mouse / trackpad. Using the Tab, Shift + Tab, Arrow, Space and Enter keys, users can successfully enter and select all offered search parameters and complete a search. I will now outline an example navigation run of a user interacting with the prototype by just using the keyboard:

Keyboard Input	Input Field	Result
Tab	Subject autocomplete input	The Subject autocomplete focuses and the dropdown menu appears with options below (Computer Science, Humanities, ect.)
Arrow (Down)	Subject autocomplete's dropdown menu	The Computer Science option is focused with a dark blue border and a faint blue background.
Enter	Subject autocomplete input	Computer Science is selected, and the text "Computer Science" is entered into the Subject input.
Tab	Course Number text input	The Course Number text input field is focused.
Keystrokes (enter the numbers 3750)	Course Number text input	"3750" is entered into the Course Number text input field
Tab	Keyword text input	The Keyword text input field is focused.
Keystrokes (enter "HCI")	Keyword text input	"HCI" is entered into the Keyword text input field.
Tab	Campus autocomplete input	The Campus autocomplete input is focused, and its dropdown menu appears and shows options (Ogden, Layton, ect.)
Arrow (Down)	Campus autocomplete input	The Ogden option is focused.

Enter	Campus autocomplete input	Ogden is selected, and the text "Ogden" is entered into the Campus autocomplete input.
Tab	Buildings autocomplete input	The Buildings autocomplete input is focused, and its dropdown appears with the available Buildings options (Science and Technology Building, Elizabeth Hall, ect.)
Arrow (Down)	Buildings autocomplete input	The Science and Technology Building option is focused.
Enter	Buildings autocomplete input	The Science and Technology Building option is selected and entered into the Buildings autocomplete input.
Tab	Open Sections Only checkbox	The Open Sections Only checkbox is focused.
Space	Open Sections Only checkbox	The Open Sections Only checkbox is checked.
Tab	Search button	The Search button is focused
Enter	Search button	Form submission, and the error validation text "Network error. Please try again later." appears.

Screen Reader Transcription Simulation

My proposed prototype supports full keyboard navigation for users that are unable or do not wish to use a mouse / trackpad. Using the Tab, Shift + Tab, Arrow, Space and Enter keys, users can successfully enter and select all offered search parameters and complete a search. I will now outline an example navigation run of a user interacting with the prototype by just using the keyboard:

I will now provide the following transcript that simulates how the prototype behaves when used with the NVDA screen reader application. I will demonstrate how each component behaves when interacted with while using the NVDA screen reader as closely as possible to how a real-life user would interact with the search feature.

Bold text are actions, and italic text are the screen reader's audio transcriptions.

User clicks on Subject autocomplete input, enters C and selects Computer Science from the dropdown list.

Weber State Registration Search.

Google Chrome window.

Main landmark.

Course Registration region.

Subject, combo box, expanded, required, has autocomplete, editable. Type or select a subject from the suggestions. Use the arrow keys to navigate and enter to select. Subject is required. Please select a subject. Enter subject blank.

C

Subject suggestions list.

Computer Science, one of two.

Subject, combo box, collapsed, required, has autocomplete. Type or select a subject from the suggestions. Use the arrow keys to navigate and enter to select. Subject is required. Please select a subject. Computer Science.

User presses tab to navigate to the Course Number text input field and enters the text 3750

Course Number edit. Enter the course number you want to find.

3750

User presses tab to navigate to the Keyword text input field and enters the text HCI

Keyword edit. Enter keywords to search course descriptions.

HCI

User presses tab to navigate to the Campus autocomplete input field, enters O and selects Ogden from the dropdown list.

Campus, combo box, expanded, has autocomplete, editable. Type or select a campus from the suggestions. Use the arrow keys to navigate and enter to select. Enter campus blank.

Campus suggestions list.

Ogden, one of four.

Campus, combo box, collapsed, has autocomplete, editable. Type or select a campus from the suggestions. Use the arrow keys to navigate and enter to select. Ogden.

User presses tab to navigate to the Buildings autocomplete dropdown, presses E and selects Elizabeth Hall from the dropdown list.

Buildings, combo box, expanded, has autocomplete, editable. Type or select a building from the suggestions. Use the arrow keys to navigate and enter to select. Enter building blank.

Buildings suggestions list.

Science and Technology Building, one of five.

Elizabeth Hall, two of five.

Buildings, combo box, collapsed, has autocomplete, editable. Type or select a building from the suggestions. Use the arrow keys to navigate and enter to select. Elizabeth Hall.

User presses tab to navigate to the Open Sections Only checkbox.

Sections availability group Open Sections Only. Checkbox not checked. Limit results to courses with available seats.

Checked.

User presses tab to navigate to the Search button and presses enter.

Search button.

Search started.

Network error. Please try again later.

This simulation also demonstrates that the use of the tab key correctly navigates through each element.

ARIA and Screen reader Hooks:

I've ensured that this prototype makes use of the following ARIA tags / attributes in order to fully accommodate screen-reader technologies.

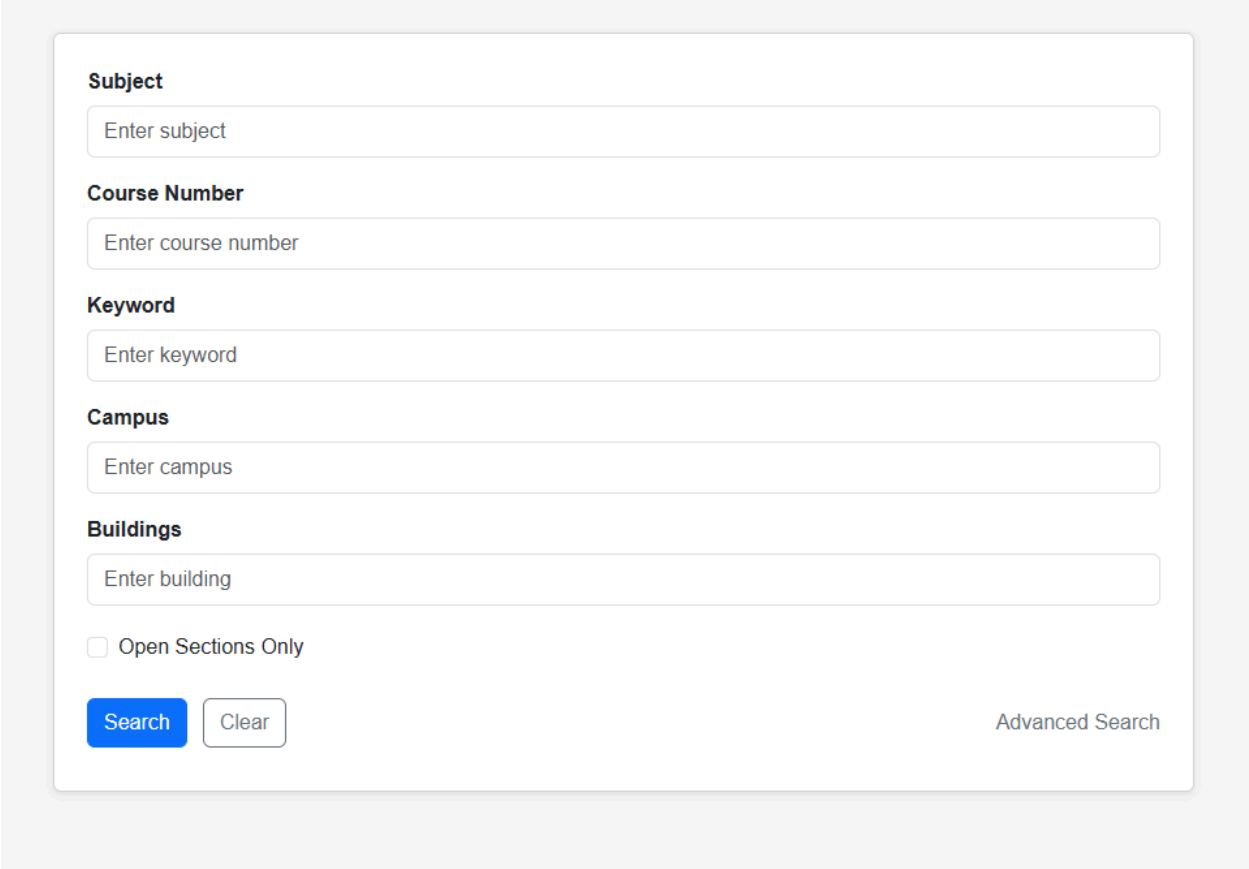
- **aria-label -** "Course registration search", "Course search form", "Subject suggestions", "Campus suggestions , "Building suggestions", "Clear all search fields"
- **aria-expanded -** Used on Subject, Campus and Building autocomplete dropdowns with true and false values to indicate open and closed states.
- **aria-controls -** Connects the Subject, Campus and Building autocomplete dropdowns to their respective results dropdowns.
- **aria-autocomplete="list"** - Informs the screenreader technology that the input provides a list of autocomplete suggestions.
- **aria-required="true"** - Used on the Subject field to highlight that the Subject field is required.
- **aria-invalid="true/false"** - Used to signal whether or not the Subject autocomplete contains an error (such as leaving it blank as it is a required field)
- **aria-selected="true/false"** - Signals which autocomplete dropdown options are currently selected or highlighted.
- **aria-describedby** - Connects each input and button's additional explanatory text.
- **aria-activedescendant** - Informs the screenreader which dropdown option is currently highlighted with the arrow keys.
- **aria-live="polite"** - Used by the Search button to announce that the search is running without interrupting the user.
- **aria-live="assertive"** - Used by the network error so that the final error text is promptly announced.

I have also made use of several aria roles and organized them alongside the categorized input fields:

- **role="combobox"** - Subject, Campus and Buildings autocomplete input fields
- **role="listbox"** - Subject, Campus and Buildings dropdown option menus
- **role="option"** - Each individual option in the dropdown menus
- **role="status"** - Search loading spinner
- **role="alert"** - Network error text warning

WCAG 2.1 AA Standards Compliance

I've designed my prototype to be in compliance with WCAG 2.1 AA standards and I've strived to maintain the 4:5:1 contrast ratio against all text and backgrounds. I have specifically colored the input labels black with bold lettering upon high-contrast white, as well as the placeholder texts within the inputs to be grey enough to read clearly, while appearing lighter and more distinguishable from the respective black bold input labels. Additionally, I've colored the Search button to be primary blue, and it stands out as the most noticeable and attention-drawing call-to-action across the entire form. The Clear button matches the white background but is colored with a grey text and outline, to distinguish its importance from the Search button. Finally, the Advanced Search link button is a similar grey text so as to not draw attention away from the rest of the interface, as it is a less-used button to expand additional optional search inputs.



Post-Mortem Reflection

How did explicitly defining every component state (Hover, Focus, Active, Loading, Error) during the specification phase prevent dynamic usability bugs that static wireframes completely fail to address?

Static wireframes are just that - static. While wireframes do a great job in mapping out workflows and outlining an implementation, an interactive prototype gives you the ability to test-drive an interface and get realistic, real-time feedback on what the user will experience on their end. Static wireframes give default appearances of layouts and certain states, but oftentimes do not account for more subtle features such as hover over / hover off animations, how UI elements respond to focus / drag / click / ect and feedback from signal UI such as loading spinners and error messages. Also, a prototype walkthrough can help you discover the emotional reaction your interface emits (i.e. comfort from the ease of use, frustration of inputs not working) - something that static wireframes simply cannot capture.

Describe a scenario in your prototype where system latency or asynchronous data processing required specialized micro-interaction feedback (e.g., loading skeletons or aria-live announcements). How does your design keep the user informed and prevent repeated submission errors?

The blue circular loading spinner that appears after clicking the Search button handles system latency and async data processing very well. The user receives immediate feedback that their input has been successfully received, validated and is being processed. The Search button is also disabled while the system is processing the input and loading the data, which prevents a potentially frustrating situation where the user may accidentally resubmit their results and trigger an unnecessary second search. The animation is also subtle and sleek, and without it, users may grow wary that the system is not working after submitting their search.